

Advanced Data Management

Chapter 4 — Class Examples (live demos): operators

Fall 2026

Run on `adm_lab`. The aim: make the executor's choices visible — which join, which sort, which aggregation — and how memory and size change them.

Results shown are from a representative run (customers $\approx$ 20,000 rows, orders $\approx$ 500,000 rows). Cost estimates are trimmed for width; exact timings and row counts will differ on your machine.

1. Example 1 — read a whole plan

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.name, SUM(o.amount) AS total
FROM customers c JOIN orders o ON o.cust_id = c.id
WHERE c.country = 'MA'
GROUP BY c.name
ORDER BY c.name;
```

Result

```
Sort (actual time=121.84..122.07 rows=1996 loops=1)
  Sort Key: c.name
  Sort Method: quicksort Memory: 205kB
  Buffers: shared hit=3355
  -> HashAggregate (actual time=120.93..121.31 rows=1996 loops=1)
    Group Key: c.name
    Batches: 1 Memory Usage: 721kB
    -> Hash Join (actual time=4.02..105.66 rows=50112 loops=1)
      Hash Cond: (o.cust_id = c.id)
      -> Seq Scan on orders o (actual time=0.01..37.95 rows=500000
        loops=1)
      -> Hash (actual time=3.91..3.92 rows=2000 loops=1)
        Buckets: 2048 Batches: 1 Memory Usage: 117kB
        -> Seq Scan on customers c (actual time=0.01..3.52 rows
          =2000 loops=1)
          Filter: (country = 'MA'::bpchar)
          Rows Removed by Filter: 18000
Planning Time: 0.412 ms
Execution Time: 122.38 ms
```

What to point out. Walk it bottom-up and name every node: the scans, the **Hash Join**, the **Sort** or **HashAggregate**. Identify the *pipeline breaker* — the operator that must consume all its input first.

2. Example 2 — force each join algorithm

```
-- default choice
EXPLAIN ANALYZE
SELECT * FROM customers c JOIN orders o ON o.cust_id = c.id;

SET enable_hashjoin = off;      -- push it to merge / nested-loop
EXPLAIN ANALYZE
SELECT * FROM customers c JOIN orders o ON o.cust_id = c.id;

SET enable_mergejoin = off;     -- now nested-loop
EXPLAIN ANALYZE
SELECT * FROM customers c JOIN orders o ON o.cust_id = c.id;

RESET ALL;
```

Result — default (hash join)

```
Hash Join (actual time=8.91..212.74 rows=500000 loops=1)
  Hash Cond: (o.cust_id = c.id)
  -> Seq Scan on orders o (actual time=0.01..38.20 rows=500000 loops=1)
  -> Hash (actual time=8.72..8.73 rows=20000 loops=1)
        Buckets: 32768 Batches: 1 Memory Usage: 1350kB
        -> Seq Scan on customers c (actual time=0.01..3.10 rows=20000 loops=1)
Execution Time: 231.52 ms
```

Result — enable_hashjoin = off (merge join)

```
Merge Join (actual time=247.30..402.81 rows=500000 loops=1)
  Merge Cond: (c.id = o.cust_id)
  -> Index Scan using customers_pkey on customers c (actual time=0.02..6.44
        rows=20000 loops=1)
  -> Sort (actual time=247.21..331.58 rows=500000 loops=1)
        Sort Key: o.cust_id
        Sort Method: external merge Disk: 8184kB
        -> Seq Scan on orders o (actual time=0.01..36.87 rows=500000 loops=1)
Execution Time: 431.90 ms
```

Result — enable_mergejoin = off as well (nested loop)

```
Nested Loop (actual time=0.03..612.44 rows=500000 loops=1)
  -> Seq Scan on customers c (actual time=0.01..2.95 rows=20000 loops=1)
  -> Index Scan using orders_cust_id_idx on orders o
        (actual time=0.004..0.027 rows=25 loops=20000)
        Index Cond: (cust_id = c.id)
Execution Time: 641.03 ms
```

Summary: hash $\approx$ 232 ms merge $\approx$ 432 ms nested loop $\approx$ 641 ms — same 500,000 rows each time.

What to point out. Same result three ways, three costs. This is exactly the “no single best join” point from the slides — and why the optimizer must choose.

3. Example 3 — external sort vs. in-memory sort

```
SET work_mem = '64kB';
EXPLAIN ANALYZE SELECT * FROM orders ORDER BY amount;    -- spills to
disk

SET work_mem = '256MB';
EXPLAIN ANALYZE SELECT * FROM orders ORDER BY amount;    -- stays in
memory
RESET work_mem;
```

Result — work_mem = 64kB

```
Sort (actual time=291.62..371.18 rows=500000 loops=1)
  Sort Key: amount
  Sort Method: external merge  Disk: 8192kB          <- small work_mem
  -> Seq Scan on orders (actual time=0.01..34.81 rows=500000 loops=1)
Execution Time: 389.47 ms
```

Result — work_mem = 256MB

```
Sort (actual time=142.31..171.94 rows=500000 loops=1)
  Sort Key: amount
  Sort Method: quicksort  Memory: 71680kB          <- large work_mem
  -> Seq Scan on orders (actual time=0.01..33.97 rows=500000 loops=1)
Execution Time: 186.02 ms
```

What to point out. Watch the **Sort Method** line flip. Small memory forces the external merge sort from the lecture; more memory removes the disk passes. Memory literally changes the algorithm.

4. Example 4 — hash vs. sort aggregation

```
EXPLAIN ANALYZE SELECT country, count(*) FROM customers GROUP BY country
;    -- few groups
EXPLAIN ANALYZE SELECT name, count(*) FROM customers GROUP BY name;
    -- many groups
```

Result — GROUP BY country (few groups)

```
HashAggregate (actual time=6.18..6.20 rows=12 loops=1)
  Group Key: country
  Batches: 1  Memory Usage: 24kB
  -> Seq Scan on customers (actual time=0.01..1.94 rows=20000 loops=1)
Execution Time: 6.29 ms
```

Result — GROUP BY name (many groups)

```
GroupAggregate (actual time=18.07..24.91 rows=19104 loops=1)
  Group Key: name
  -> Sort (actual time=18.05..19.63 rows=20000 loops=1)
      Sort Key: name
      Sort Method: quicksort  Memory: 1623kB
      -> Seq Scan on customers (actual time=0.01..2.02 rows=20000 loops=1)
Execution Time: 25.58 ms
```

What to point out. Few groups → **HashAggregate** (a tiny hash table). Many groups → the planner may sort first. Same “sort vs. hash” choice as joins, one level up.

5. Example 5 — size flips the join choice

```
-- very selective: a few customers -> nested-loop / index
EXPLAIN ANALYZE
SELECT * FROM customers c JOIN orders o ON o.cust_id = c.id WHERE c.id <
    10;

-- the whole thing -> hash join
EXPLAIN ANALYZE
SELECT * FROM customers c JOIN orders o ON o.cust_id = c.id;
```

Result — `c.id < 10` (selective)

```
Nested Loop  (actual time=0.03..0.21 rows=225 loops=1)
->  Index Scan using customers_pkey on customers c  (actual time=0.01..0.02
      rows=9 loops=1)
      Index Cond: (id < 10)
->  Index Scan using orders_cust_id_idx on orders o
      (actual time=0.004..0.017 rows=25 loops=9)
      Index Cond: (cust_id = c.id)
Execution Time: 0.26 ms
```

Result — whole join

```
Hash Join  (actual time=8.91..212.74 rows=500000 loops=1)
  Hash Cond: (o.cust_id = c.id)
->  Seq Scan on orders o  (actual time=0.01..38.20 rows=500000 loops=1)
->  Hash  (actual time=8.72..8.73 rows=20000 loops=1)
      Buckets: 32768  Batches: 1  Memory Usage: 1350kB
      -> Seq Scan on customers c  (actual time=0.01..3.10 rows=20000 loops=1)
Execution Time: 231.52 ms
```

What to point out. Nothing changed but the number of rows involved — and the optimizer picked a different join. This is the perfect hand-off into Chapter 5.

`RESET ALL;` between demos so a left-over `SET` does not confuse the next plan.